

Trivy Security Dashboard

A comprehensive Flask-based dashboard for visualizing Trivy security scan results from CycloneDX files stored in Nexus Repository.

Overview

This dashboard provides real-time security insights by:

- Fetching CycloneDX SBOM files from Nexus Repository
- Parsing vulnerability data from multiple projects and scans
- Displaying interactive charts and analytics
- No separate database required - data is fetched directly from Nexus

Features

Application Level Views

- **Vulnerability Trends:** Track vulnerabilities over time
- **Risk Assessment:** Critical, High, Medium, Low severity breakdown
- **Component Analysis:** Most vulnerable components and licenses
- **Compliance Reports:** Security posture across all applications

Pipeline Level Views

- **Build History:** Security scan results per pipeline execution
- **Quality Gate Status:** Pass/fail trends and thresholds
- **Scan Performance:** Scan duration and efficiency metrics
- **Remediation Tracking:** Vulnerability resolution progress

Multi-Scan Analytics

- **Cross-Project Comparison:** Security posture across different projects
- **Dependency Analysis:** Common vulnerable dependencies
- **Security Metrics:** KPIs and security score calculations
- **Alert Dashboard:** Critical issues requiring immediate attention

Architecture

```
dashboard/
├── app.py           # Main Flask application
├── requirements.txt # Python dependencies
├── config.py        # Configuration management
├── services/        # Backend services
│   ├── nexus_client.py # Nexus API integration
│   ├── cyclonedx_parser.py # SBOM parsing logic
│   └── analytics.py    # Data analysis and metrics
```

```
├── templates/           # HTML templates
│   ├── base.html       # Base template
│   ├── dashboard.html  # Main dashboard
│   ├── project.html    # Project-specific view
│   └── scan.html        # Individual scan details
├── static/             # Static assets
│   ├── css/            # Custom stylesheets
│   ├── js/             # JavaScript and charts
│   └── img/            # Images and icons
└── utils/              # Utility functions
    ├── cache.py        # Caching mechanism
    └── helpers.py      # Helper functions
```

Quick Start

1. Install Dependencies

```
cd dashboard
pip install -r requirements.txt
```

2. Configure Nexus Connection

```
# Set environment variables or edit config.py
export NEXUS_URL="http://your-nexus.company.com:8081"
export NEXUS_USERNAME="your-username"
export NEXUS_PASSWORD="your-password"
export NEXUS_REPOSITORY="trivy-sbom"
```

3. Run Dashboard

```
python app.py
```

4. Access Dashboard

Open browser to <http://localhost:5000>

Dashboard Views

Main Dashboard

- Security overview across all projects
- Trending vulnerability counts
- Critical alerts and notifications

- Quick access to problem areas

Project View

- Project-specific security metrics
- Build history and scan results
- Dependency vulnerability analysis
- Remediation recommendations

Scan Details

- Individual scan breakdown
- Component-level vulnerability details
- SBOM metadata and analysis
- Export and sharing options

Configuration

The dashboard automatically discovers and processes:

- **CycloneDX Files:** Stored in Nexus with pattern `{groupId}/{artifactId}/{version}/`
- **Project Metadata:** Extracted from SBOM metadata and Nexus artifact properties
- **Scan History:** Based on build numbers and timestamps
- **Vulnerability Data:** Parsed from CycloneDX vulnerability components

Data Flow

1. **Discovery:** Scans Nexus repository for CycloneDX artifacts
2. **Fetching:** Downloads SBOM files based on filters and timeframes
3. **Parsing:** Extracts vulnerability, component, and metadata information
4. **Analysis:** Calculates trends, metrics, and security scores
5. **Visualization:** Renders interactive charts and dashboards
6. **Caching:** Stores processed data temporarily for performance

UI Features

- **Responsive Design:** Works on desktop, tablet, and mobile
- **Interactive Charts:** Hover, zoom, and drill-down capabilities
- **Real-time Updates:** Automatic refresh of data from Nexus
- **Export Options:** Download reports as PDF, Excel, or JSON
- **Dark/Light Mode:** User preference themes
- **Search & Filter:** Find specific projects, scans, or vulnerabilities

Metrics & Analytics

Security Metrics

- Vulnerability density (vulnerabilities per component)
- Mean time to remediation (MTTR)

- Security debt (total vulnerability exposure)
- Risk score calculation

Trend Analysis

- Vulnerability introduction rate
- Remediation velocity
- Quality gate pass/fail trends
- Component age and update frequency

Compliance Reporting

- Policy compliance percentage
- Audit trail and change tracking
- Risk assessment summaries
- Executive dashboards

Security Features

- **Read-only Access:** Dashboard only reads from Nexus, no modifications
- **Authentication:** Optional integration with corporate SSO
- **Access Control:** Project-based permission filtering
- **Audit Logging:** Track dashboard usage and access patterns

Customization

Adding New Charts

Add chart configurations in `static/js/charts.js`:

```
const customChart = {  
  type: 'bar',  
  data: processedData,  
  options: chartOptions  
};
```

Custom Metrics

Extend `services/analytics.py`:

```
def calculate_custom_metric(s bom_data):  
    # Your custom calculation logic  
    return metric_value
```

Theme Customization

Modify `static/css/custom.css` for branding and styling.

API Endpoints

The dashboard exposes REST APIs for integration:

- `GET /api/projects` - List all projects
- `GET /api/project/{id}/scans` - Get scans for a project
- `GET /api/scan/{id}/vulnerabilities` - Get vulnerabilities for a scan
- `GET /api/metrics/summary` - Get security metrics summary

Troubleshooting

Common Issues

1. **Nexus Connection:** Verify URL, credentials, and network access
2. **No Data:** Check CycloneDX files exist in configured repository
3. **Performance:** Adjust caching settings for large datasets
4. **Charts Not Loading:** Ensure JavaScript is enabled

Debug Mode

Enable debug logging:

```
export FLASK_DEBUG=1
export LOG_LEVEL=DEBUG
python app.py
```

Dependencies

- **Flask:** Web framework
- **Requests:** HTTP client for Nexus API
- **Pandas:** Data analysis and manipulation
- **Plotly:** Interactive charting
- **Bootstrap:** UI framework
- **CycloneDX:** SBOM parsing library

Deployment

Docker Deployment

```
docker build -t trivy-dashboard .
docker run -p 5000:5000 \
  -e NEXUS_URL=http://nexus.company.com:8081 \
  -e NEXUS_USERNAME=username \
  -e NEXUS_PASSWORD=password \
  trivy-dashboard
```

Production Considerations

- Use WSGI server (Gunicorn, uWSGI)
- Configure reverse proxy (Nginx, Apache)
- Set up SSL/TLS certificates
- Implement monitoring and logging
- Configure backup and disaster recovery